

INTERVIEW DOCS

Java Interview Questions

Frequently Asked Core Java Questions with Detailed Answers

Java Agentic AI — Knowledge Base Reference

1. Language Fundamentals

Q1. Why is Java platform-independent?

Java source code compiles to an intermediate bytecode format (.class files) rather than native machine code. Any machine with a JVM for its OS/architecture can execute that same bytecode, so the same compiled artifact runs unchanged everywhere a JVM exists — 'Write Once, Run Anywhere'.

Q2. What is the difference between == and equals()?

== compares references for objects (are they the exact same object in memory?) or raw values for primitives. equals() compares logical/content equality and can be overridden to define what 'equal' means for a given class (String, Integer, and custom classes override it; the Object default is reference equality).

Q3. Why is String immutable in Java?

- Security — Strings are used for classloading, file paths, network connections; mutability could allow tampering after validation.
- Thread-safety — immutable objects are automatically safe to share across threads with no synchronization.
- Hashcode caching — since content never changes, String can cache its hashCode, making it very efficient as a HashMap key.
- String pool — immutability is what makes literal sharing/interning safe (two variables can safely point at the same object).

Q4. Difference between abstract class and interface?

An abstract class can hold instance state, a constructor, and a mix of abstract/concrete methods, but supports only single inheritance. An interface defines a pure capability contract, supports multiple implementation, and (since Java 8) can carry default/static method bodies, but cannot hold instance state beyond public static final constants. Use an abstract class for a shared base with common state; use an interface for a capability unrelated classes can plug into.

Q5. What is the difference between checked and unchecked exceptions?

Checked exceptions (IOException, SQLException) are verified by the compiler — a method must either catch them or declare them via throws. Unchecked exceptions (RuntimeException and its subclasses, like NullPointerException) are not compiler-enforced and typically represent programming bugs rather than recoverable external conditions.

2. Collections & Generics

Q6. HashMap vs TreeMap vs LinkedHashMap?

Map	Ordering	Performance
HashMap	No guaranteed order	O(1) average for get/put

Map	Ordering	Performance
LinkedHashMap	Insertion order (or access order if configured)	$O(1)$ average, slight overhead over HashMap
TreeMap	Sorted by key (natural or Comparator)	$O(\log n)$ — backed by a red-black tree

Q7. How does HashMap handle collisions?

Each bucket holds a linked list of entries that hash to the same index. On collision, the new entry is appended to that bucket's chain. Since Java 8, once a single bucket's chain grows beyond 8 entries (with at least 64 total buckets), it's converted to a red-black tree, bounding worst-case lookup at $O(\log n)$ instead of $O(n)$.

Q8. What is type erasure?

Generics exist only at compile time for type checking; the compiler erases type parameters, replacing them with their bound (or `Object`), and inserts casts where needed. This is why you can't do `'new T[10]'` or `'obj instanceof List'` — that type information simply isn't present at runtime.

Q9. ArrayList vs LinkedList — when would you choose each?

`ArrayList` backs onto a resizable array — $O(1)$ random access, but $O(n)$ insert/delete in the middle due to element shifting. `LinkedList` is a doubly-linked list — $O(n)$ random access, but $O(1)$ insert/delete once you're already positioned at a node. Choose `ArrayList` for read-heavy, index-based access; `LinkedList` for frequent insertions/removals at known positions (though `ArrayDeque` often outperforms `LinkedList` in practice for queue/stack use cases).

3. OOP & Design

Q10. Can a static method be overridden?

No. Static methods are resolved at compile time based on the reference (declared) type — this is called method hiding, not overriding. If a subclass declares a static method with the same signature, calling it through a superclass reference invokes the superclass's version, unlike instance method overriding which resolves polymorphically at runtime based on the actual object type.

Q11. What is the diamond problem, and how does Java avoid it?

The diamond problem arises when a class inherits from two sources that both define a conflicting method, and it's ambiguous which implementation should apply. Java avoids it for classes by disallowing multiple class inheritance entirely. For interfaces with default methods, if a class implements two interfaces with the same default method signature, the compiler forces the implementing class to explicitly override and resolve the conflict (optionally delegating to `InterfaceName.super.method()`).

Q12. What design principle does 'favor composition over inheritance' refer to?

It suggests building complex behavior by combining simpler objects as fields (HAS-A) rather than through class hierarchies (IS-A). Composition avoids the fragile base class problem, allows runtime flexibility (swap out a component), and avoids deep, rigid inheritance chains that become hard to change safely.

4. Multithreading

Q13. Difference between synchronized and volatile?

synchronized ensures mutual exclusion — only one thread executes the protected block/method at a time — providing both atomicity and visibility. volatile only guarantees visibility (reads always see the latest write) and prevents certain instruction reordering, but does NOT make compound operations like increment atomic.

Q14. What causes deadlock, and how can you avoid it?

Deadlock happens when two or more threads each hold a lock the other needs and neither releases — a circular wait. Avoid it by always acquiring multiple locks in the same global order across all threads, using tryLock() with timeouts to back off instead of blocking indefinitely, and keeping critical sections as small as possible.

Q15. Runnable vs Callable?

Runnable.run() returns void and cannot throw checked exceptions. Callable.call() returns a value of type V and can throw checked exceptions — it's used with ExecutorService.submit(), which returns a Future representing the pending result.

5. JVM Internals

Q16. What's the difference between the stack and the heap?

The heap is shared across all threads and stores all objects/arrays, managed by the garbage collector. Each thread has its own stack, storing method call frames — local variables, parameters, and partial results for that thread only; stack memory is automatically reclaimed when a method returns.

Q17. What triggers a StackOverflowError vs an OutOfMemoryError?

StackOverflowError occurs when a thread's call stack exceeds its fixed size, typically from unbounded or excessively deep recursion. OutOfMemoryError occurs when the heap (or metaspace) cannot allocate more memory for new objects/classes because it's full and the garbage collector can't reclaim enough space.

Q18. What is JIT compilation?

The Just-In-Time compiler identifies frequently executed ('hot') bytecode paths at runtime and compiles them directly to optimized native machine code, combining the portability of an interpreter with near-native performance for long-running applications.

6. Modern Java

Q19. What problem do Optional and records solve, respectively?

Optional makes 'this value might be absent' explicit in a method's return type, reducing surprise NullPointerExceptions from callers who forget to null-check. Records eliminate boilerplate for immutable data carriers — the compiler auto-generates the constructor, accessors, equals(), hashCode(), and toString() from the field list.

Q20. What are virtual threads and why do they matter?

Introduced in Java 21, virtual threads are lightweight threads managed by the JVM rather than mapped 1:1 to OS threads. They make blocking-style I/O code scale to enormous levels of concurrency (potentially millions of threads) without the memory and context-switching cost of traditional platform threads, competing with reactive programming for I/O-bound workloads while keeping simple, sequential code style.

7. Rapid-Fire Recap Table

Question	One-line Answer
Can constructors be private?	Yes — used in Singleton and static factory patterns
Can an interface have a constructor?	No — only classes can
Is Java pass-by-value or pass-by-reference?	Always pass-by-value; object references are passed by value
Default value of a boolean field?	false
Can you overload the main method?	Yes, but the JVM only ever calls the standard public static void main
What does 'final' mean for a variable/method/class?	Non-reassignable / non-overridable / non-subclassable respectively